

C# Assignment - Theory Answers

Comments in Code

Single-line comments use `//` while multi-line comments use `/* */`.

Example:

Single-line: `// Calculates the sum`

Multi-line:

```
/*
```

Stores numbers and prints their sum

```
*/
```

Shortcut in Visual Studio:

Comment selected code: `Ctrl + K + C`

Uncomment selected code: `Ctrl + K + U`

Fixing Code Errors

Correct code:

```
int x = 10;
```

```
int y = 5;
```

```
Console.WriteLine(x + y);
```

The original code had:

1. Invalid assignment of string to int.
2. console should be Console.
3. Variable y was not declared.

Runtime Error vs Logical Error

Runtime Error:

Occurs during program execution.

Example: dividing by zero.

Logical Error:

The program runs but produces incorrect results.

Example: using `+` instead of `*` in a calculation.

Naming Conventions

Example declarations:

```
string fullName;
```

```
int age;
```

```
double monthlySalary;
```

```
bool isStudent;
```

PascalCase improves readability, consistency, and code maintenance.

Value Types and Reference Types

Value types store data directly in stack memory.
Each variable has its own copy.

Reference types store references in stack memory and actual objects in heap memory.
Multiple references can point to the same object.

Modulo Operator Output

Code:

```
int a = 2, b = 7;  
Console.WriteLine(a % b);
```

Output: 2
Because 2 is smaller than 7, the remainder remains 2.

Logical AND vs Bitwise AND

&& checks logical conditions and stops evaluation if the first condition is false.
& compares bits and evaluates both conditions regardless.

Casting

Explicit casting is required from double to int because data loss may occur.
The decimal part is removed during conversion.

Parse Exception

Invalid input may cause FormatException.
It can be handled using try-catch blocks.

Prefix and Postfix Increment

Prefix (++x) increases the value before usage.
Postfix (x++) uses the value first then increases it.

Code:

```
int x = 5;  
int y = ++x + x++;
```

Final value of x = 7.

Stack and Heap Allocation

Value types are usually stored in stack memory for fast access.

Reference types are stored in heap memory while references are stored in stack memory.

Compiled vs Interpreted Languages

Compiled languages translate code into machine language before execution.

Interpreted languages execute code line by line.

C# is mainly a compiled language. It compiles to Intermediate Language (IL) then runs using the CLR.

Implicit, Explicit, Convert, and Parse

Implicit Casting:

Automatic conversion without data loss.

Explicit Casting:

Manual conversion when data loss may happen.

Convert:

Provides safe conversions between types.

Parse:

Converts strings into specific data types.

Managed Code

Managed code runs under the control of the CLR.

The CLR handles memory management, garbage collection, and security.

Struct vs Class

Structs are value types while classes are reference types.

Structs are lightweight and stored mostly in stack memory.

Classes support inheritance and heap allocation.